IEEE CSTAM 3.0 × FELCLOUD

Resilient IP Optimizer

System architecture, OpenStack high availability and the gateway data flow

Source: docs/architecture and docs/diagrams

Repository revision df10b63

01 October 2026

Contents

1 **System architecture**

What it does, in one paragraph · Components · Networks and addressing · Design decisions · Technology choices · Capacity and limits · Security in brief · What is verified, and what is not

2 **OpenStack high availability**

Targets · OpenStack resources behind the design · How the gateway failover works · Split brain · Everything else that can fail · Findings from the real FelCloud cloud · Known limits and risks

3 **Gateway data flow**

Lane 1: the request path · Lane 2: the configuration path · Lane 3: signals and telemetry · Data inventory · Failover, as seen by this diagram

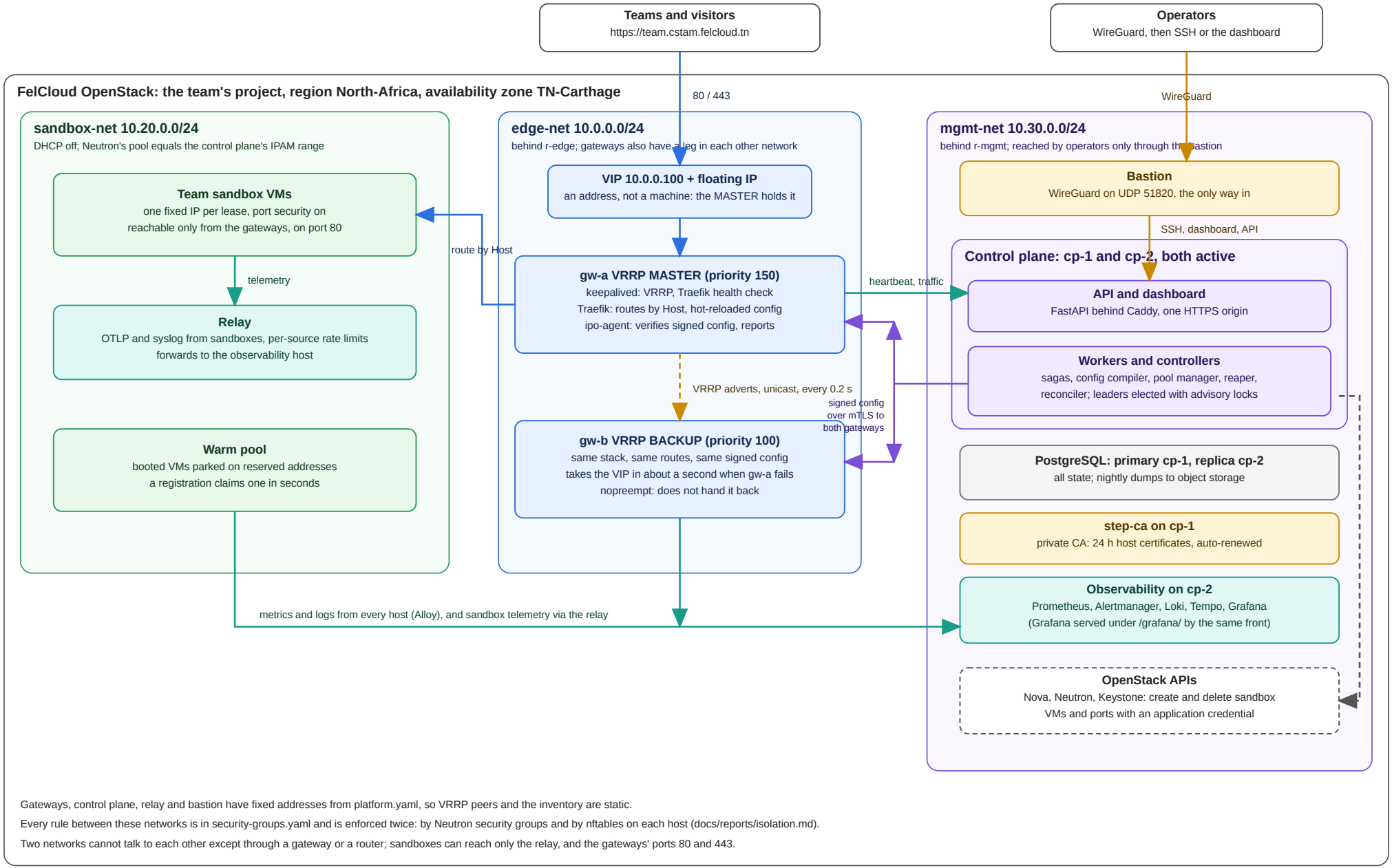
PART 1

System architecture

The Resilient IP Optimizer gives every team in the CSTAM 3.0 challenge an isolated sandbox VM on FelCloud OpenStack and a stable public subdomain (`<team>.cstam.felcloud.tn`), all behind one floating IP. A pair of gateways shares that address so the platform keeps serving when a gateway VM or its proxy dies, and a control plane creates, routes, repairs and removes sandboxes automatically.

System architecture

The Resilient IP Optimizer on FelCloud OpenStack: one floating IP, a VRRP gateway pair, a control plane that provisions and routes, and the isolated sandbox VMs behind them.



Gateways, control plane, relay and bastion have fixed addresses from platform.yaml, so VRRP peers and the inventory are static.

Every rule between these networks is in security-groups.yaml and is enforced twice: by Neutron security groups and by nftables on each host (docs/reports/isolation.md).

Two networks cannot talk to each other except through a gateway or a router; sandboxes can reach only the relay, and the gateways' ports 80 and 443.

Source: `docs/diagrams/system_architecture.py` *(SVG:* `system-architecture.svg` *).*

What it does, in one paragraph

A team registers (`POST /v1/teams`). The control plane claims a pre-booted VM from a warm pool, adds a route for `<team>.cstam.felcloud.tn` , signs a new gateway configuration, and pushes it to both gateways. Each gateway's agent verifies, validates, swaps in and probes the new config without restarting Traefik; once both gateways acknowledge it, the team's subdomain answers from its sandbox. When the lease expires (or the team is deleted) the route is removed first, then the VM and port, then the address is quarantined and returned to the pool. If the active gateway dies, the other takes over the virtual IP in about a second.

Components

Component	Runs on	Technology	Responsibility
Gateway pair (`gw-a` , `gw-b`)	edge-net, sandbox-net, mgmt-net	keepalived, Traefik 3, `ipo-agent`	Hold the VIP (VRRP), terminate HTTP, route by `Host` to sandboxes
ipo-agent	each gateway	Go, standard library only	Accept signed config, verify, validate, swap, probe, roll back; heartbeat; report traffic
Control plane (`cp-1` , `cp-2`)	mgmt-net	Python 3.12, FastAPI, psycopg	API, sagas, config compiler, warm pool, reaper, reconciler, audit
Dashboard	served by the control plane's front	React 19, TypeScript, Tailwind 4	Staff console and a per-team page
PostgreSQL	cp-1 primary, cp-2 replica	PostgreSQL 18	All state; triggers enforce the lease state machine and the audit chain
step-ca	cp-1	smallstep CA	24 h host certificates for mTLS, renewed automatically
Observability	cp-2	Prometheus, Alertmanager, Loki, Tempo, Grafana	Metrics, alerts, logs, traces, dashboards
Relay	sandbox-net and mgmt-net	Grafana Alloy	Takes telemetry from untrusted sandboxes, rate-limited per source
Bastion	mgmt-net, floating IP	WireGuard	The only way in for operators
Sandbox VMs	sandbox-net	the baked `ipo-sandbox-vN` image	The teams' workloads, one fixed IP each

The containerised services (Traefik, the control plane and its web front, PostgreSQL, step-ca, the observability stack) run as rootless Podman Quadlets (`quadlets/`); keepalived, `ipo-agent` , Alloy and WireGuard are ordinary system services.

Networks and addressing

All addresses are fixed in `platform.yaml`, so VRRP peers, the Ansible inventory and the firewall rules are static.

Network	CIDR	Router	Holds
`edge-net`	10.0.0.0/24	`r-edge` (external gateway)	the VIP `10.0.0.100`, gateway edge ports `.11` and `.12`
`sandbox-net`	10.20.0.0/24, DHCP off	`r-edge`	gateway legs `.2`, `.3`; relay `.4`; sandboxes `.10` - `.109` (the IPAM range)
`mgmt-net`	10.30.0.0/24	`r-mgmt` (external gateway)	bastion `.10`, gateways `.11` / `.12`, `cp-1` `.21`, `cp-2` `.22`, relay `.31`

Neutron's allocation pool on sandbox-net is exactly the control plane's IPAM range, so the cloud never hands out an address the control plane also leases.

Design decisions

The numbered ADRs in `docs/adr` record the reasoning; the ones that shape everything:

1. **One source of truth per concern.** `platform.yaml` (sizes, addresses, timings, flavors) and `security-groups.yaml` (every allowed flow) are read by Pulumi, Ansible, the control plane, the agents and the tests. A rule or address is changed in one place and a test fails if a layer disagrees (ADR 3).
2. **The database enforces the lease state machine.** A `BEFORE UPDATE` trigger rejects illegal transitions, so no application bug or second writer can double-lease an address (ADR 2).
3. **Long operations are sagas.** Registration and teardown record each step; a dead worker's job is resumed by another, a failing step is undone in reverse, and steps are idempotent.
4. **Config is signed and the agent is the last line of defence.** The control plane signs every gateway config with Ed25519; the agent verifies it and independently re-validates the content (only Host rules, backends only inside the sandbox CIDR), so a compromised control plane cannot point a gateway at an arbitrary address.
5. **Gateways do not need the control plane to serve.** They keep the last verified config and restore it on restart; the control plane is on the config path, never the request path.
6. **Isolation is enforced twice.** Neutron security groups and per-host nftables are generated from the same file and tested separately and together (isolation results).
7. **Containers on the host network, rootless** (ADR 4), so the host firewall is exactly what guards each port.
8. **The cloud is behind an interface.** The control plane talks to OpenStack through a `Cloud` protocol with an SDK implementation and an in-memory fake, which is what makes the whole flow testable without a cloud.

Technology choices

Choice	Why
Traefik file provider	Hot reload from an atomically replaced file; no API to secure

Choice	Why
keepalived, unicast VRRP	Standard, sub-second failover; multicast is often filtered on Neutron
Ed25519 signatures	Small, fast, no parameters to get wrong; the public key is the only thing agents hold
Postgres `LISTEN/NOTIFY` and advisory locks	Debounced recompile on route change; leader election without another service
Podman Quadlet	Declarative units with health checks and auto-update rollback, no daemon
Pulumi (Python)	One language for the infra and its tests; policy-as-code with CrossGuard
Go for the agent	One static binary with no runtime on a security-sensitive host

Capacity and limits

The FelCloud project allows 50 instances, 128 cores, 256 GB RAM, 10 floating IPs and 100 security-group rules (survey). The platform itself uses 6 VMs, 10 cores, 12 GB, 2 floating IPs and 38 rules. The sandbox pool holds 100 addresses, but the instance limit is the real ceiling: about 44 sandboxes, warm pool included. Registration refuses to drop below a reserve of free addresses (`pool.reserve_free_ips`) but does not know about the instance limit; a burst past it fails at Nova.

Security in brief

Roles `viewer` < `operator` < `admin` plus a separate `team` role that no staff route admits; argon2id passwords; short-lived JWTs; mTLS between control plane and agents with 24 h certificates; secrets as Podman secrets; a hash-chained audit log. Network isolation, the threat table and key handling are in security.md.

What is verified, and what is not

Verified	How
Control plane flows (registration, teardown, reconciler, roles, config pipeline)	300+ unit and integration tests against a real PostgreSQL
Gateway failover, config hot-reload, traffic accounting, split-brain detection	real keepalived, Traefik and `ipo-agent` in a rootless namespace lab, under load
Network isolation, both enforcement layers and each alone	namespace lab with 1,683 probes per pass
Every Ansible template, Quadlet and observability config	read by the real program at the pinned version
`pulumi up` and `destroy`	run against the real FelCloud project (86 resources, VMs ACTIVE in about 45 s)

Not verified	Why
Ansible roles on real VMs; VRRP between two real OpenStack VMs	no configured hosts yet; the floating IPs attach, but nothing answers behind them until the roles run through the bastion

Not verified	Why
`podman auto-update` rollback, step-ca enrollment end to end, Postgres replication between hosts, WireGuard	no Podman runtime or second host was available
Behaviour with real teams, and the instance limit under a burst	not exercised

Repository map

Path	Contents
`control-plane/`	the Python service, migrations, Containerfile, 300+ tests
`gateway-agent/`	the Go agent and a dev gateway
`dashboard/`	the React console
`infra/pulumi/`	day-0 infrastructure, CrossGuard policies, mocked tests
`ansible/`	day-1 roles and playbooks, inventory, filter plugin
`quadlets/`	day-2 container units
`observability/`	alert rules, dashboard generator and dashboards
`tests/`	chaos lab, isolation lab, deploy checks, the demo
`docs/`	this documentation
`platform.yaml`, `security-groups.yaml` (+ schemas)	the single sources of truth

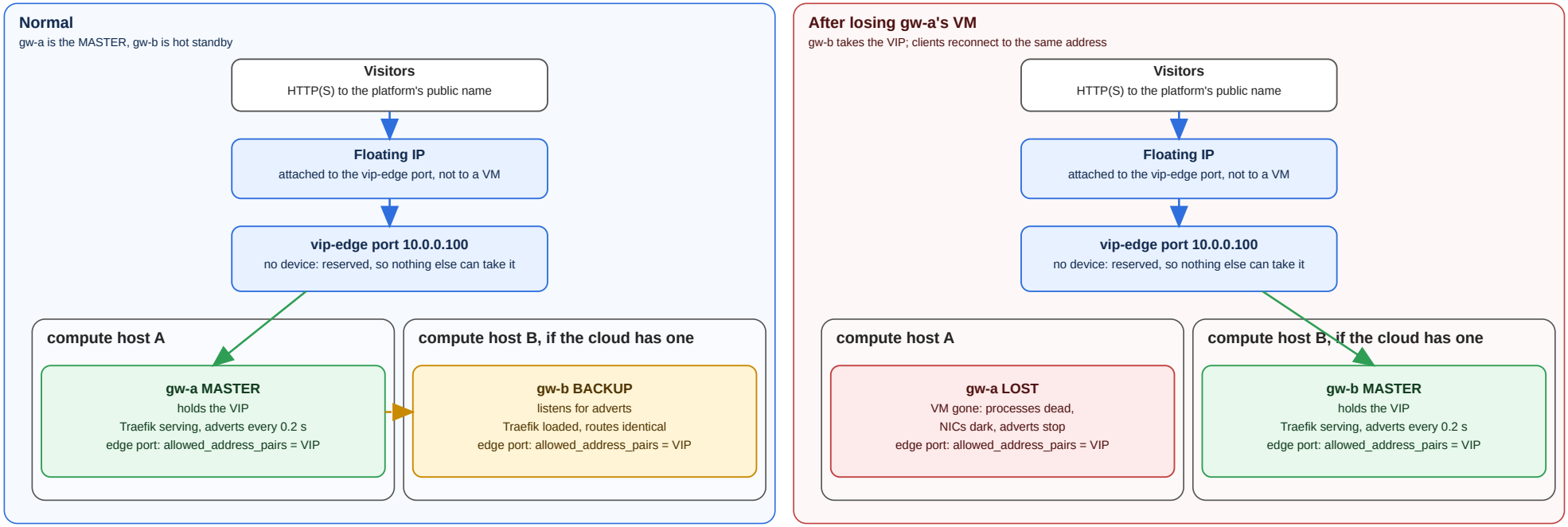
PART 2

OpenStack high availability

How the platform stays up on OpenStack when a gateway, a proxy, the control plane or the database fails: what each failure costs, how it is detected, and what the evidence is.

OpenStack high availability

One floating IP and one VIP port; two gateways that can each hold the VIP; everything the failover needs from Neutron and Nova, and what is redundant elsewhere.



VRRP adverts, unicast, every 0.2 s, gw-a to gw-b (dashed)

t+0 gw-a dies; about 0.6 s later gw-b misses three adverts, becomes MASTER, adds the VIP, sends gratuitous ARPs
Measured under 50 new connections/s: worst gap 0.52 to 0.66 s over 11 VM-death runs (Traefik crash: 1.2 to 1.7 s; keepalived stop: 0.12 s).

What the handover needs from OpenStack

Allowed address pair
Both gateways' edge ports list the VIP, so port security lets them send and receive it

A device-less VIP port
Holds 10.0.0.100 and the floating IP, so the address survives either VM

Unicast VRRP
Multicast is often filtered on Neutron; the security group admits VRRP between gateways

FAULT releases at once
Check script with no weight: two failed checks = FAULT = priority-0 advert, no wait

nopreempt
The old MASTER returns as BACKUP, so a recovery is not a second outage

What is redundant, and what is not

verified in the namespace lab with the real software; not yet on real OpenStack VMs

Gateways
VRRP pair, anti-affinity
(soft: see the HA document)
loss: about 0.6 s

Control plane
two active replicas, leaders by advisory lock; gateways serve without it

PostgreSQL
streaming replica on cp-2; promote_replica.yml is manual; writes pause meanwhile

ipo-agent
last-known-good config kept; restart resumes it; systemd restarts it

Sandbox VM
one team's outage only; reconciler repairs drift; warm pool refills

CA, observability
single instance each: 24 h certificates give a window; not redundant

Source: `docs/diagrams/openstack_ha.py`.

Targets

Objective	Target	Measured (lab)
Gateway VM dies	VIP moves in about 2 s	**0.52 to 0.66 s** worst request gap (11 runs)
Traefik crashes or hangs	about 4 s	**1.2 to 1.7 s** (SIGKILL, SIGTERM, 12 runs)
keepalived stopped cleanly	immediate	**0.12 s**
Manual failover through the control plane	under 1.5 s	**0.12 s**
Old MASTER comes back	no second outage	**0 s**, no flap (nopreempt)
Both gateways claim MASTER (partition)	detected within seconds	alarm in about 5 s; one MASTER again after healing
Control plane down	gateways keep serving	verified: config pipeline works with no control plane

Method: the lab runs real keepalived 2.4, Traefik 3.7 and `ipo-agent`, and sends 50 new TCP connections per second through the VIP; a "gap" is the longest stretch in which no request succeeded. See `tests/chaos/` and `uv run python -m chaos.run`. These numbers come from Linux bridges in network namespaces, **not from OpenStack VMs** (see limits).

OpenStack resources behind the design

Built by `infra/pulumi` from `platform.yaml` and `security-groups.yaml`.

Need	OpenStack object	Detail
Three isolated networks	`edge-net`, `sandbox-net`, `mgmt-net` + subnets	sandbox-net: DHCP off, allocation pool = the IPAM range
Reach the internet	routers `r-edge` (edge + sandbox) and `r-mgmt`	external gateway on `INTERNET`, SNAT
An address that survives a gateway	port `vip-edge` (no device) + floating IP	the VIP is reserved, so nothing else can take it
Let either gateway use the VIP	`allowed_address_pairs` on both gateways' edge ports	otherwise port security drops VIP traffic
No spoofing	`port_security_enabled` on every port and network	enforced by the CrossGuard policy `port-security-on`
Network policy	9 security groups, 38 rules, remote-group references	one Neutron group per spec group and network, e.g. `sg-gateway.edge`
Keep a pair apart	server groups `gateways-anti`, `control-plane-anti`	`soft-anti-affinity` by default (see risks)
Boot with no metadata service	config drive	sandbox-net has no DHCP and no metadata route

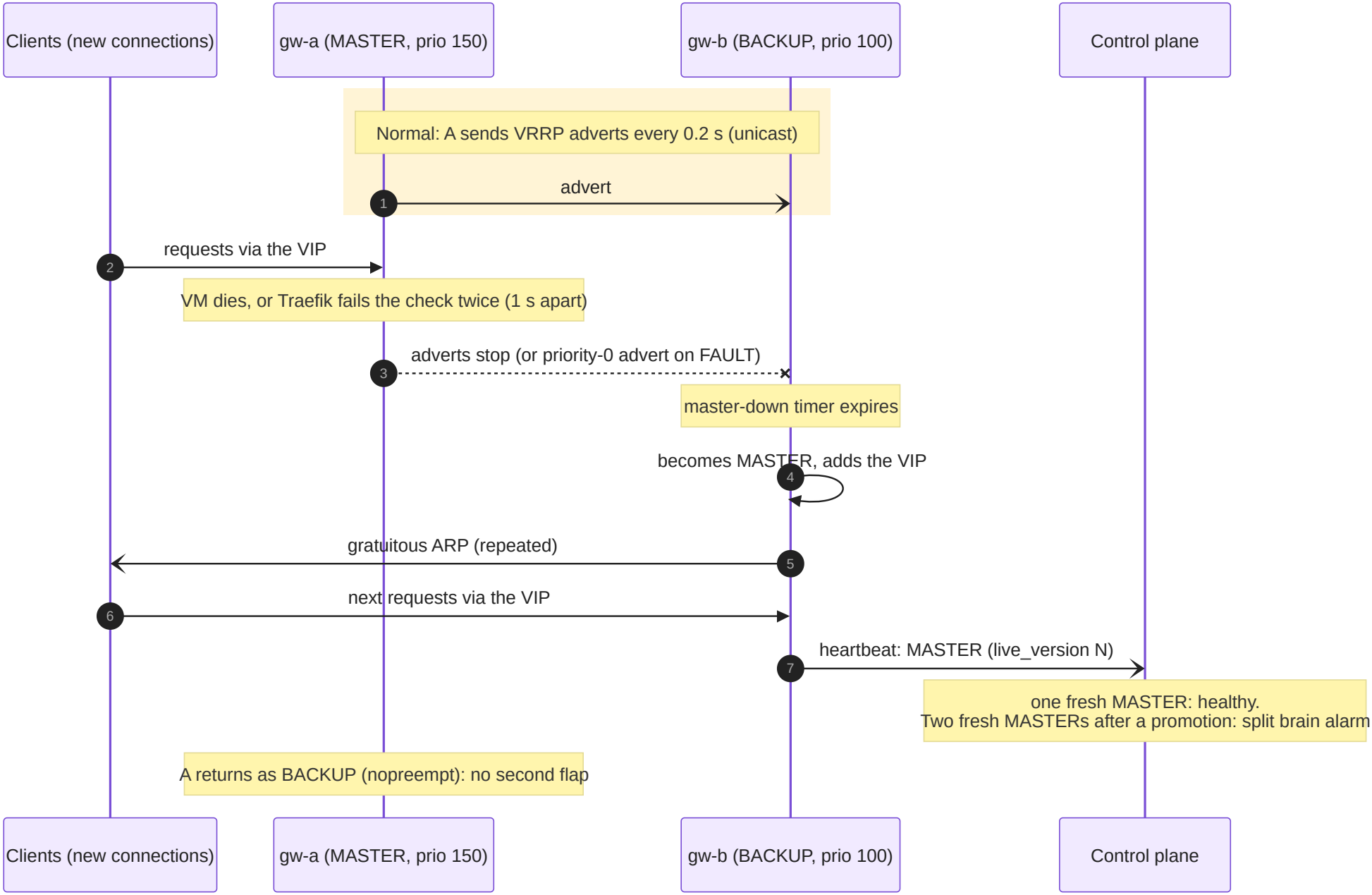
Need	OpenStack object	Detail
Inventory by role	server metadata `role=<name>`	enforced by CrossGuard `vm-has-role`

How the gateway failover works

Both gateways run keepalived in `BACKUP` state with `nopreempt`; gw-a has priority 150 and becomes MASTER first, gw-b has 100.

1. **Adverts.** The MASTER sends a VRRPv3 advert to its peer every 0.2 s, unicast (multicast is often filtered on Neutron). The security group admits VRRP only between the gateways.
2. **Detection, machine failure.** The BACKUP declares the MASTER down after missing three adverts (about 0.6 s plus a priority-dependent skew), then takes the VIP.
3. **Detection, proxy failure.** keepalived runs a check against Traefik's ping endpoint every 1 s; two failures in a row put the instance in `FAULT`. The check has no weight, so `FAULT` makes keepalived send a priority-0 advert and release the VIP at once, instead of lowering its priority and waiting for a preempt that `nopreempt` forbids.
4. **Takeover.** The new MASTER adds `10.0.0.100/24`, sends gratuitous ARPs (after 1 s, repeated five times, refreshed every 30 s) so neighbours and Neutron converge, and its notify hook writes `MASTER` to `/run/ipo/vrrp_state`.
5. **Report.** `ipo-agent` reads that file and heartbeats the control plane every 5 s.
6. **Recovery.** The old MASTER returns as BACKUP. Because of `nopreempt` the VIP stays where it is, so recovery is not a second outage.

A deliberate failover (`POST /v1/gateways/failover`, admin) writes a non-zero value to keepalived's track file through the agent (`POST /fault`), which has the same effect as step 3. `DELETE /fault` (or removing `/run/ipo/fault`) lifts it. Restarting the agent does not.


Clients (new connections)
gw-a (MASTER, prio 150)
gw-b (BACKUP, prio 100)
Control plane
Normal: A sends VRRP adverts every 0.2 s (unicast)
advert
1
requests via the VIP
2
VM dies, or Traefik fails the check twice (1 s apart)
adverts stop (or priority-0 advert on FAULT)
3
master-down timer expires
becomes MASTER, adds the VIP
4
gratuitous ARP (repeated)
5
next requests via the VIP
6
heartbeat: MASTER (live_version N)
7
one fresh MASTER: healthy.
Two fresh MASTERs after a promotion: split brain alarm
A returns as BACKUP (nopreempt): no second flap
Clients (new connections)
gw-a (MASTER, prio 150)
gw-b (BACKUP, prio 100)
Control plane

Split brain

If the gateways cannot hear each other (a partition) both can become MASTER and both add the VIP. The platform detects this rather than preventing it:

- Each agent reports its VRRP state and the time it last changed (`vrrp_since`).
- The control plane counts gateways whose latest heartbeat is MASTER and fresh (within 15 s). Two qualify as a split brain only if both have reported since the newest promotion: a gateway that died as MASTER stops reporting, so the survivor taking over is not an alarm.
- It raises `ipo_gateway_masters > 1` , increments `ipo_split_brain_total` , emits a `gateway.split_brain` event (and `…_resolved` later), shows a banner in the dashboard and fires the `IPOGatewaySplitBrain` alert after 5 s.
- Healing the partition lets VRRP re-elect by priority; the lab verifies a single MASTER and a single VIP holder afterwards.

Everything else that can fail

Failure	Effect on teams	Detection	Recovery
Gateway VM or Traefik	under 2 s of failed new connections	VRRP, check script	automatic; Quadlet and systemd restart the process
`ipo-agent` crashes	none; Traefik keeps its config	the Gateways page shows a stale heartbeat; a restart shows as flapping	systemd restarts it; it resets `live.yml` to its last known good
Control plane replica	none; the other replica serves	`/readyz` , Prometheus	automatic; saga jobs resume on the survivor
Both control-plane replicas	no new registrations or teardowns; **existing sites keep serving**	`/readyz` , alerts	restart; the reconciler repairs drift on return
PostgreSQL primary	control plane stops writing; sites keep serving	`/readyz`	`playbooks/promote_replica.yml` (manual) – runbook
A team's sandbox VM	that team only	reconciler: lease `leased` but VM gone	team marked failed, teardown started, alert `alert.vm_gone`
Bad config pushed	none	agent probe fails	agent rolls back to last known good; `IPOConfigRolledBack`
Forged or corrupted config	none	signature check	rejected 403; `IPOConfigSignatureRejected` pages
step-ca (cp-1)	none for 24 h	renewal timer errors	restore cp-1; certificates are valid 24 h and renew when 8 h remain
Observability (cp-2)	none	–	restart; not redundant
OpenStack API outage	no new VMs; existing traffic unaffected	saga failures	sagas retry with backoff; pool backs off

Control plane internals that make it safe to run twice

- **Workers** claim jobs with row locks; a job whose worker has been silent for 60 s is picked up by another and resumed from the last completed step.
- **Controllers** (compiler, pool manager, reaper, reconciler) each run in one replica at a time, chosen by a Postgres advisory lock; if that replica dies, the other takes the lock.
- **Everything that matters is in PostgreSQL**; replicas hold no state.

Findings from the real FelCloud cloud

Two things the first real `pulumi up` showed that no test could:

- **Hard anti-affinity failed.** The four VMs in `anti-affinity` server groups went to `ERROR` while the two without a group booted, which fits a single compute host in TN-Carthage. The default is now `soft-anti-affinity`, which separates the pair when it can. **If the cloud has one host, a host failure takes both gateways at once**; failover protects against VM and process failure, not host failure. Ask FelCloud how many hosts back the zone.
- **Floating IPs need the subnet named, and a routed subnet.** Allocation without a subnet returned `ExternalIpAddressExhausted` although the pool had room; naming the external subnet works. Attaching a floating IP to the device-less VIP port then needs that port's subnet to be on a router with an external gateway already, so the association waits for the router interface. With both in place the floating IP attaches to `vip-edge` and goes `ACTIVE`. Whether traffic then reaches whichever gateway holds the VIP is **not yet tested** (it needs the gateways configured).

Known limits and risks

Risk	Status
VRRP between two real OpenStack VMs is **untested**: the lab uses Linux bridges. Neutron must deliver VIP traffic (arriving through the floating IP) to whichever port lists it as an allowed address pair and learn the move from the gratuitous ARPs. The floating IP itself does attach to the VIP port.	needs the Ansible roles run on the built VMs
One compute host would defeat the gateway pair (soft anti-affinity)	ask FelCloud; documented
PostgreSQL promotion is manual; there is no automatic fencing beyond the playbook stopping the old primary	accepted for the challenge; runbook exists
step-ca and observability are single instances	accepted; 24 h certificate window
Agents log in to the API with an operator account; a compromised gateway would hold operator rights	a narrower `gateway` role would close it
Instance quota (50) caps sandboxes at about 44	documented; ask for more before a load test

PART 3

Gateway data flow

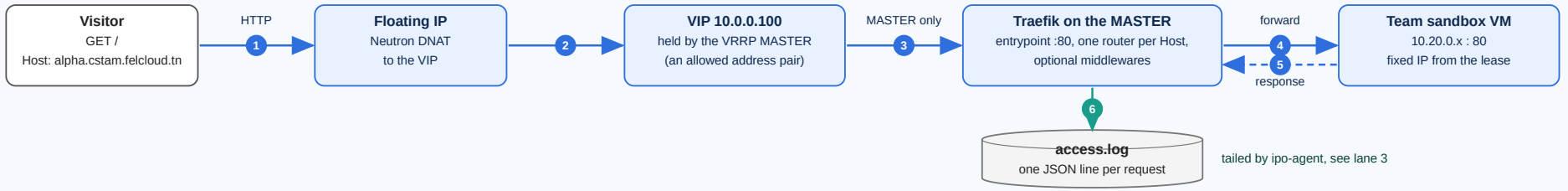
What moves through a gateway, in which direction, in what format, and what happens when each part fails. The diagram has three lanes; each section below walks one lane using the same numbers and letters.

Gateway data flow

What moves through a gateway, and where it comes from: requests, configuration, and the signals and telemetry that flow back. gw-b runs the identical stack as the VRRP BACKUP.

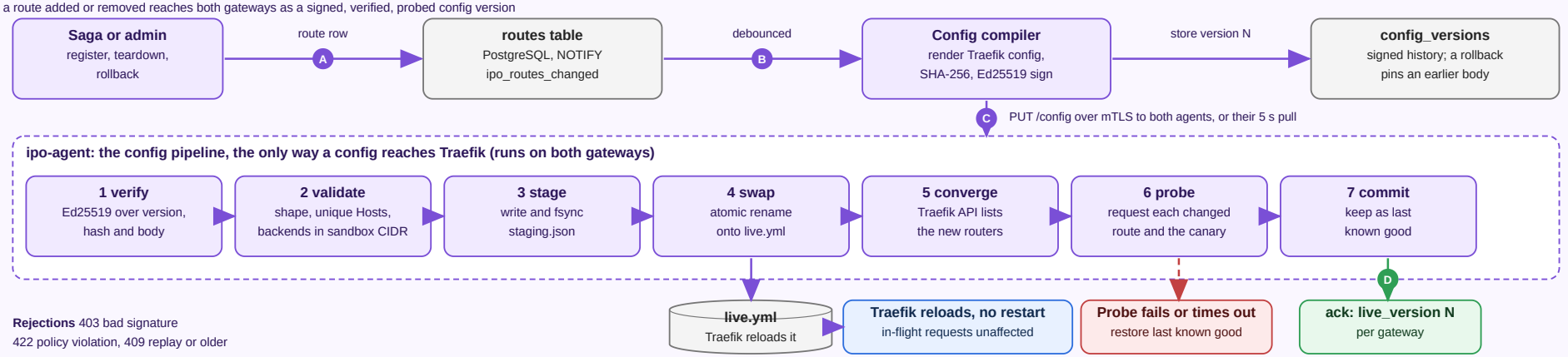
1 Request path

a visitor reaching a team's sandbox, through whichever gateway holds the VIP



2 Configuration path

a route added or removed reaches both gateways as a signed, verified, probed config version

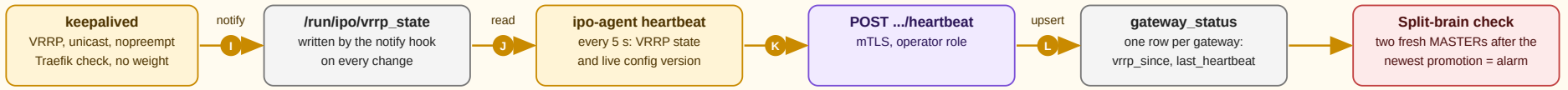


Version rules: a version goes live only when both gateways ack it; a rejected one is never retried (a transport failure is, with backoff); rollback re-issues an earlier body under a higher number. **If the control plane is down** both gateways keep serving the last version they verified.

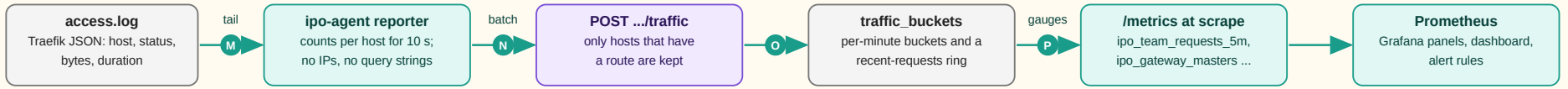
3 Signals and telemetry

what the gateways report back, and what the control plane does with it

VRRP state and liveness



Traffic and metrics



Source: `docs/diagrams/gateway_data_flow.py` *(SVG:* `gateway-data-flow.svg` *).*

A gateway is a VM with three network legs running keepalived, Traefik and `ipo-agent`. There are two, `gw-a` and `gw-b`; only the VRRP MASTER receives requests, but both receive every config version, so the standby is always ready.

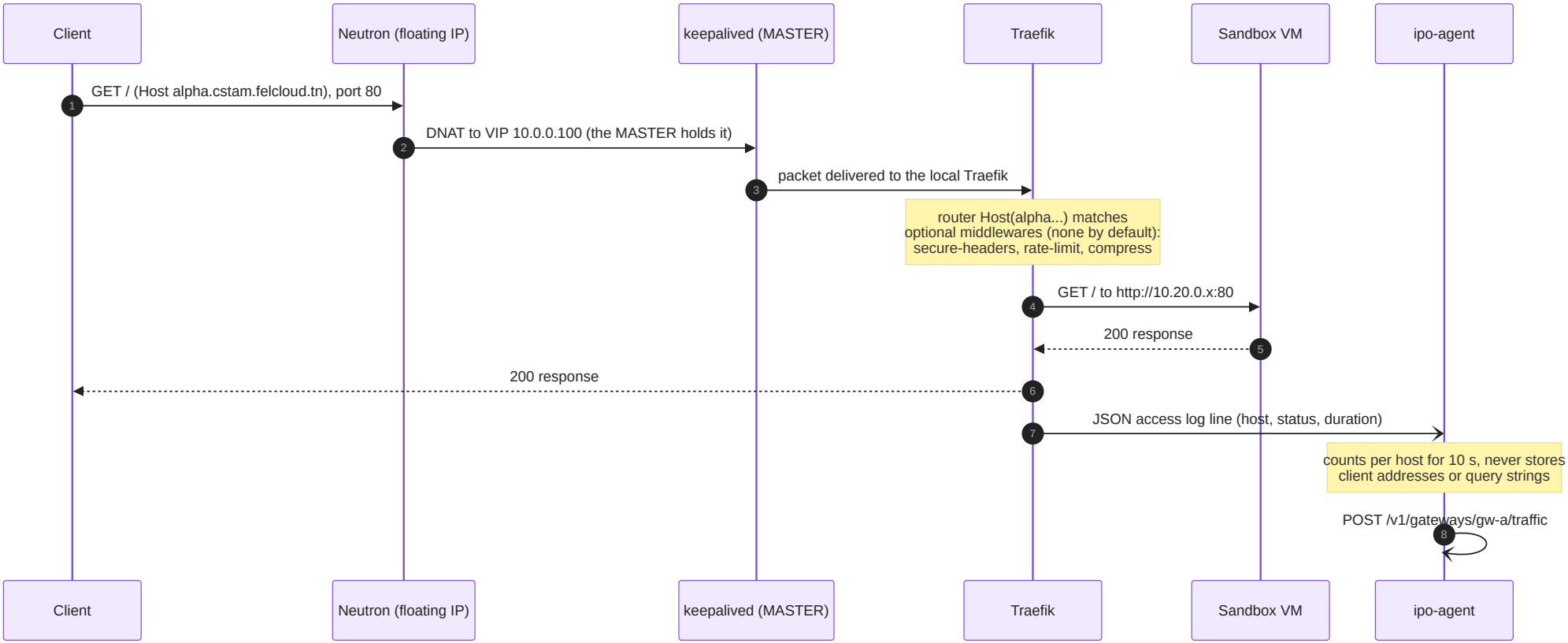
Lane 1: the request path

A visitor's request for `alpha.cstam.felcloud.tn`:

Step	What happens
1	The browser resolves the team's name to the platform's floating IP and sends HTTP to port 80
2	Neutron translates the floating IP to the VIP `10.0.0.100` (the `vip-edge` port)
3	The VIP is held by the VRRP MASTER only, so the packet reaches that gateway's Traefik
4	Traefik's `web` entry point matches the router whose rule is `Host(`alpha.cstam.felcloud.tn`)` and forwards to `http://<team ip>:80` over sandbox-net, applying the route's optional middlewares
5	The sandbox's response returns along the same path
6	Traefik appends one JSON line to `access.log` (host, method, path, status, bytes, duration)

Properties worth knowing:

- Each gateway has its own legs; the sandbox sees the request come from the gateway's sandbox-net address (`10.20.0.2` or `.3`), which identifies which gateway served it (the demo uses this to show failover).
- A policy route on the gateways (`gateway_network` role) makes replies to edge-sourced traffic leave by the edge port. Without it, a request that reached a gateway through the router would be answered from the sandbox leg with an edge source address, and port security would drop it.
- Middlewares are opt-in per route and none by default: `secure-headers` (HSTS, nosniff, frame-deny), `rate-limit` (100 req/s average, burst 50), `compress`.
- Hosts that have no route get Traefik's own `404`. A route to a sandbox that is down gets `502`.


Client
Neutron (floating IP)
keepalived (MASTER)
Traefik
Sandbox VM
ipo-agent
1
GET / (Host alpha.cstam.felcloud.tn), port 80
2
DNAT to VIP 10.0.0.100 (the MASTER holds it)
3
packet delivered to the local Traefik
router Host(alpha...) matches
optional middlewares (none by default):
secure-headers, rate-limit, compress
4
GET / to http://10.20.0.x:80
5
200 response
6
200 response
7
JSON access log line (host, status, duration)
counts per host for 10 s, never stores
client addresses or query strings
8
POST /v1/gateways/gw-a/traffic
Client
Neutron (floating IP)
keepalived (MASTER)
Traefik
Sandbox VM
ipo-agent

Lane 2: the configuration path

A route changing (a team activates, is deleted, or an admin rolls back) becomes a new signed config version on both gateways.

Step	What happens
A	A saga (or an admin action) inserts or deletes a row in `routes`
B	PostgreSQL `NOTIFY ipo_routes_changed` wakes the compiler, which debounces bursts (0.1 s)
C	The compiler renders Traefik's dynamic config, computes its SHA-256, signs `version + hash + body` with Ed25519, stores it in `config_versions`, and `PUT`s it to **both** agents over mTLS. Each agent also polls `GET /v1/config/latest` every 5 s, which covers a missed push
1 to 7	Inside the agent, in order: **verify** the signature (403 if bad), **validate** the content, **stage** to `staging.json` (fsync), **swap** by atomic rename onto `live.yml`, **converge** (wait until Traefik's API lists the new routers), **probe** (request every route changed since the live config, plus the canary, through Traefik), **commit** (hard-link as `lkg-N.json`, drop older ones)
D	The agent answers `200` with its `live_version`. The version becomes `live` only when both gateways acknowledge it; if either refuses (policy 422), it is marked `rejected`

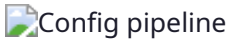
The compiled config is a pure function of the routes: the same routes always produce the same bytes, so an unchanged state never creates a new version. For two teams, one with a middleware:

```
{"http": {
  "middlewares": {"secure-headers": {"headers": {"contentTypeNosniff": true, "frameDeny": true,
                                                 "stsSeconds": 31536000}}},
  "routers": {
    "ipo-canary":               {"entryPoints": ["web"], "rule": "Host(`canary.cstam.felcloud.tn`)", "serv
ice": "ipo-canary"},
    "t-alpha-cstam-felcloud-tn": {"entryPoints": ["web"], "rule": "Host(`alpha.cstam.felcloud.tn`)", "serv
ice": "t-alpha-cstam-felcloud-tn"},
    "t-bravo-cstam-felcloud-tn": {"entryPoints": ["web"], "middlewares": ["secure-headers"],
                                  "rule": "Host(`bravo.cstam.felcloud.tn`)", "service": "t-bravo-cstam-fel
cloud-tn"}},
  "services": {
    "ipo-canary":               {"loadBalancer": {"servers": [{"url": "http://127.0.0.1:8082"}]}},
    "t-alpha-cstam-felcloud-tn": {"loadBalancer": {"servers": [{"url": "http://10.20.0.11:80"}]}},
    "t-bravo-cstam-felcloud-tn": {"loadBalancer": {"servers": [{"url": "http://10.20.0.12:80"}]}}}}}
```

(It is written as JSON with sorted keys; JSON is valid YAML, which is what Traefik's file provider reads from `live.yml`.)

What the agent refuses (policy 422), whatever the control plane signed: a config that is not the expected shape, two routers for one host, a rule that is not exactly one `Host(`name`)`, a backend that is not inside the sandbox CIDR (or the one loopback canary address), a middleware that is not on the allow-list, or a router that references a missing service.

The canary is a route (`canary.<domain>`) to a responder inside the agent (`127.0.0.1:8082`). Probing it through Traefik proves the new config was loaded and routes end to end even when no team has changed.


Config pipeline

Config pipeline

Failure behaviour

Event	Result
Signature does not verify	`403`; nothing touches `live.yml`; `IPOConfigSignatureRejected` pages
Content breaks the rules	`422` with the reason; not retried
Same version and hash again	acknowledged as a no-op; an older or equal version with a different hash is a `409` replay
Probe fails or Traefik does not converge in time	`live.yml` is restored from the last known good file, `422`, `IPOConfigRolledBack` warns; the control plane marks that version `rejected` until the next route change
Agent killed mid-swap	on start it resets `live.yml` to the newest `lkg-*` file (or an empty config)
An agent is unreachable	the push retries with backoff (5 attempts); the reconciler re-pushes a gateway that is behind
Control plane down	nothing changes; both gateways keep the last verified config
Operator rolls back	the control plane re-issues the earlier body under a **new, higher** version, so agents never see time go backwards

Lane 3: signals and telemetry

VRRP state and liveness (I to L)

Step	What happens
I	keepalived runs its notify hook on every state change; the hook atomically writes `MASTER`, `BACKUP`, `FAULT` or `STOPPED` to `/run/ipo/vrrp_state`
J	`ipo-agent` reads that file (two consecutive identical reads before a change counts, so a half-written file cannot cause a spurious state)
K	Every 5 s the agent `POST`s `/v1/gateways/<name>/heartbeat` with `{vrrp_state, live_version}`, authenticated as an operator
L	The control plane upserts `gateway_status` (state, live version, `vrrp_since`, `last_heartbeat`). The split-brain check reads it; the dashboard's Gateways page and the `ipo_gateway_masters` gauge show it

Traffic and metrics (M to P)

Step	What happens
M	The agent tails Traefik's JSON access log and counts requests per host: total, status class (2xx to 5xx), bytes, summed duration. Only host, method, path **without the query string**, status, size and duration are read; client addresses and headers are never read
N	Every 10 s it `POST`s a batch to `/v1/gateways/<name>/traffic` (up to 500 hosts and 200 recent requests; excess hosts are dropped and counted). A failed POST is re-queued within a memory bound. The canary's own probes are ignored

Step	What happens
O	The control plane keeps only hosts that have a route (so a scanner sending random `Host` headers cannot grow the tables): per-minute buckets for 26 h and a ring of the last 2,000 requests. Time is the server's, never the agent's
P	`/metrics` computes gauges from the database at scrape time, so every replica reports the truth: for example `ipo_team_requests_5m{team,class}` and `ipo_team_latency_ms_5m{team}` for teams that still have a route. Prometheus scrapes them; Grafana panels, the dashboard's Traffic page and alerts read them

Data inventory

Data	Where it lives	Written by	Kept
Routes	`routes`	sagas	while the team is active
Config versions (body, SHA-256, signature, status)	`config_versions`	compiler	forever (history, rollback source)
Live config on a gateway	`live.yml`, `lkg-N.json`	agent	newest known good only
VRRP state	`/run/ipo/vrrp_state`; `gateway_status`	keepalived hook; control plane	latest
Access log	`/var/log/traefik/access.log`	Traefik	local, rotated by the host
Traffic counters	`traffic_buckets`, `traffic_recent`	control plane, from agent batches	26 h, last 2,000
Audit trail	`audit_log` (hash-chained)	every state-changing action	forever; integrity checked by `GET /v1/audit`

Failover, as seen by this diagram

When the MASTER fails, lane 1 changes only in step 3: the VIP, and therefore the traffic, moves to the other gateway. Lanes 2 and 3 are unaffected, because both gateways already hold the same config version and both report. The new MASTER's first heartbeat tells the control plane which gateway now serves, and the traffic reporter on that gateway starts counting the requests it now receives.